
1. La Regla de Oro: La rama main es sagrada

Nunca, bajo ninguna circunstancia, subas cambios directamente a main (o master). Esta rama siempre debe contener código que funcione perfectamente y esté listo para producción.

2. El Ciclo de Trabajo Correcto

Para cada nueva funcionalidad o corrección de error, sigue este proceso:

1. **Crea una rama nueva:** Dale un nombre descriptivo.
 - `git checkout -b feature/nueva-funcionalidad`
2. **Trabaja y haz commits:** Haz cambios pequeños y frecuentes con mensajes claros.
3. **Sincroniza con el equipo:** Antes de enviar tus cambios, asegúrate de tener lo último de tus compañeros.
 - `git checkout main`
 - `git pull origin main`
 - `git checkout tu-rama`
 - `git merge main` (Esto resuelve conflictos en *tu* computadora, no en el servidor).
4. **Sube tu rama:** `git push origin feature/nueva-funcionalidad`.

3. Pull Requests (PR): El "Control de Aduana"

En lugar de fusionar tu código directamente, abres un **Pull Request** en GitHub. Esto permite que:

- Tus compañeros revisen tu código.
- Se ejecuten pruebas automáticas para ver si rompiste algo.
- Se discutan mejoras antes de que el código llegue a la rama principal.

4. Cómo evitar los "Conflictos de Fusión"

Los conflictos ocurren cuando dos personas editan la misma línea del mismo archivo al mismo tiempo. Para minimizarlos:

- **Divide el trabajo:** Intenten no trabajar en el mismo archivo simultáneamente.
 - **Pull frecuente:** Haz `git pull` seguido para estar al tanto de lo que suben los demás.
 - **Commits atómicos:** Cambios pequeños son más fáciles de fusionar que un cambio gigante de 50 archivos.
-

5. Herramientas de "Emergencia"

Si algo sale mal, no entres en pánico. Git tiene memoria de elefante:

- **git status:** Tu mejor amigo para saber dónde estás parado.
- **git checkout -- archivo:** Deshace los cambios locales en un archivo.
- **git revert [ID-del-commit]:** Crea un nuevo commit que deshace exactamente lo que hizo uno anterior, sin borrar el historial (es la forma más segura de "volver atrás" en equipo).

Resumen para el equipo:

Acción	Comando / Lugar
Empezar algo nuevo	git checkout -b nombre-rama
Guardar progreso	git commit -m "explicación"
Pedir revisión	Botón "New Pull Request" en GitHub
Limpiar errores	git revert o pedir ayuda antes de usar push --force

Un consejo final: Prohíbe el uso de git push --force en el equipo. Ese comando es el verdadero "destructor" de repositorios porque sobrescribe el historial de los demás.